

实验二：跨平台应用开发

实验项目基本信息

- 实验编号：d20301035102、d20301009202
- 学时分配：4 学时
- 实验类型：验证型
- 每组人数：6 人
- 对应课程目标：课程目标 1

🔗 核心步骤列表

📖 实验概览

- 需求分析阶段：编写在线学习辅助平台功能需求文档（登录验证、角色权限、题库管理）
- AI 辅助编码阶段：使用 TRAE 生成 Cordova 项目骨架代码并完善业务逻辑
- 测试验证阶段：编写功能测试用例，验证边界条件和考试防作弊机制
- 部署运维阶段：打包调试版本，记录多平台兼容性对比报告

📋 开发任务清单

任务阶段	主要内容	关键技术点
登录模块	HTML/CSS/JS + Cordova	localStorage、API 调用、JWT
题库管理	增删改查操作	IndexedDB、AJAX、RESTful
在线考试	定时功能、防作弊	setInterval、摄像头检测、切屏监控
AI 错题分析	错题收集与分析	本地存储、智能推荐算法

✔ 成功标准

- 登录注册功能完整，支持角色切换（普通用户/儿童/家长）
- 题库管理系统实现增删改查
- 考试模块具备计时功能和防作弊机制
- AI 错题分析功能正常运行
- 多平台打包测试通过（Android/iOS）
- 实验报告完成

实验任务与案例

核心任务

使用 Cordova 框架开发"在线学习辅助平台", 技术栈为 HTML+CSS+JavaScript, 实现登录模块、题库管理、在线考试（含定时功能）、AI 错题分析等功能, 最终打包为 Android/iOS 应用。

实战案例

开发"在线学习辅助平台", 支持三种用户角色:

- 儿童用户: 显示学习内容、课程推荐、学习任务
- 家长用户: 显示孩子学习数据、学习进度、成长报告
- 普通用户: 显示个性化学习中心、我的课程、通知公告

第一课时: 项目搭建与登录模块 (2 学时)

2.1 需求分析阶段

步骤 1: 编写需求文档

1. 功能需求

- 账号登录 (手机号/用户名)
- 密码输入
- 记住密码、自动登录选项
- 登录按钮、注册入口、忘记密码入口
- 输入验证 (不能为空、长度限制、手机号格式验证)
- 对接后端登录接口
- JWT Token 存储与自动刷新
- 用户角色 (普通用户/儿童/家长) 识别
- 用户信息本地缓存
- 多角色权限控制

2. 界面需求

- Material Design 风格
- 输入框占位提示
- 密码隐藏显示切换
- 加载状态提示
- 错误信息友好提示
- 适合不同年龄层用户的交互设计

3. 跳转需求

- 登录成功 → 根据用户角色跳转对应主页
- 儿童用户: 跳转到儿童学习主页

- 家长用户：跳转到家长管控主页
- 普通用户：跳转到学习中心主页
- 传递用户角色、ID、昵称等数据
- 登录状态持久化，下次启动自动校验

4. 题库管理需求

- 题目增删改查
- 题目分类管理（语文、数学、英语等）
- 题目难度分级（简单、中等、困难）
- 批量导入导出

5. 在线考试需求

- 计时功能（倒计时）
- 自动交卷
- 防作弊：切屏检测、摄像头监控、禁止复制粘贴

6. AI 错题分析需求

- 自动收集错题
- 错题分类统计
- 智能推荐同类题目
- 学习进度可视化

系统架构设计

本项目采用前后端分离架构，Android 端作为前端交互层，Spring Boot 作为后端服务层，MySQL 作为数据存储层，整体架构如下：

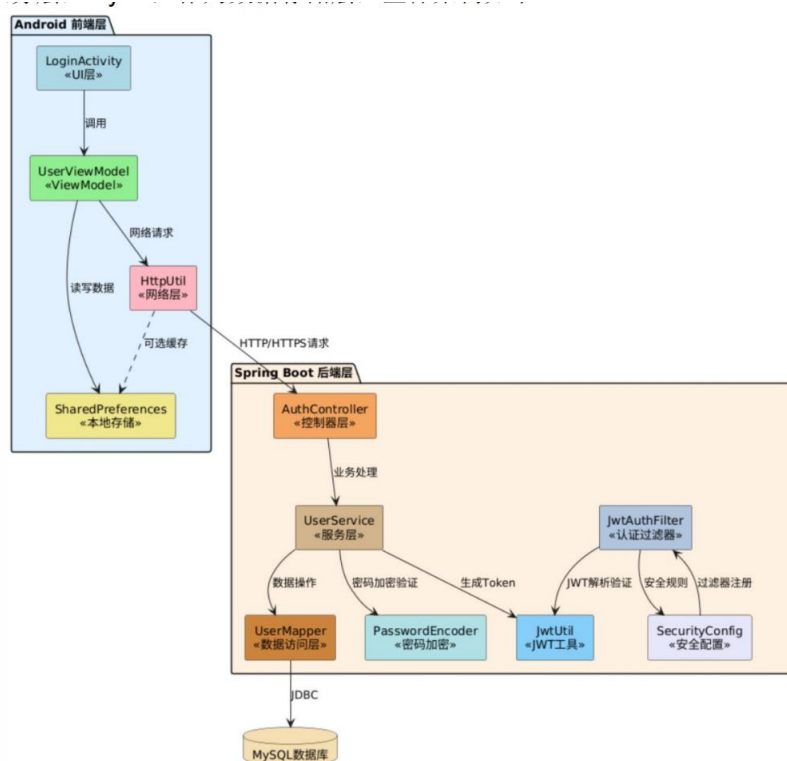


图 1. 系统整体架构图

登录流程设计

用户登录请求从 Android 端发起，经过多层验证和处理后完成身份认证，完整登录时序如下：

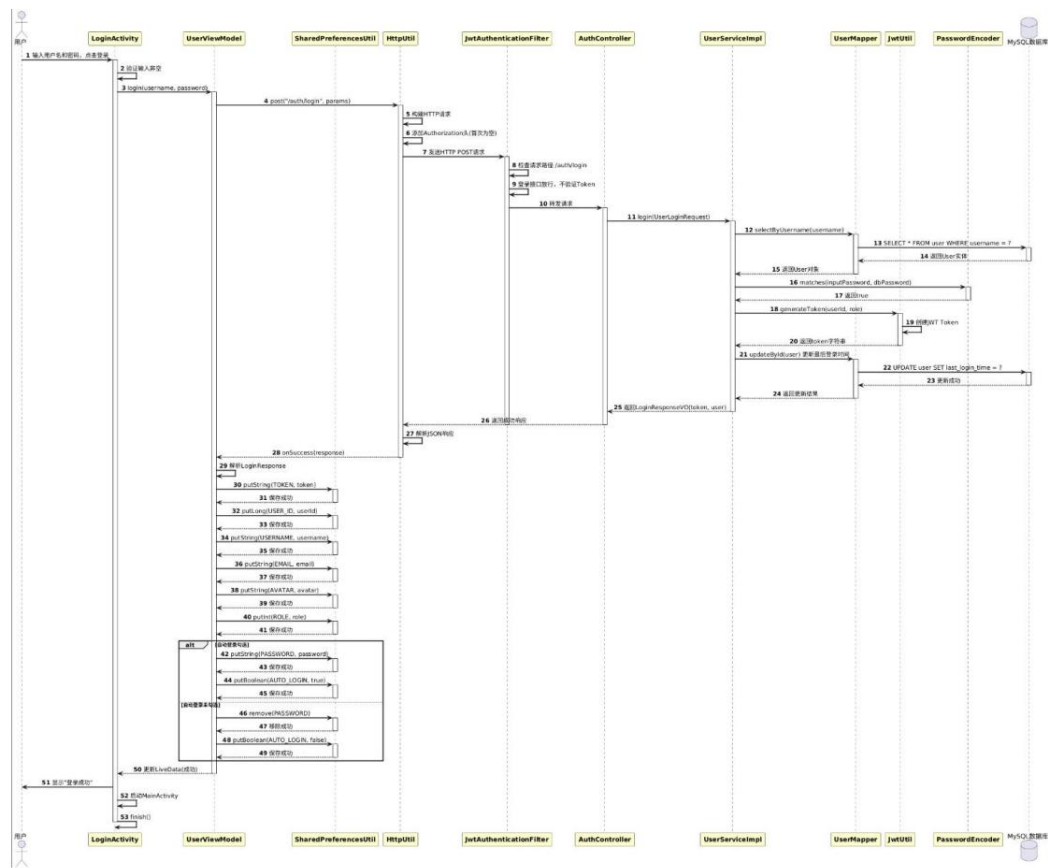


图 2. 登录时序图

核心类结构设计

系统前后端核心类设计遵循分层架构思想，各层职责明确，类关系如下

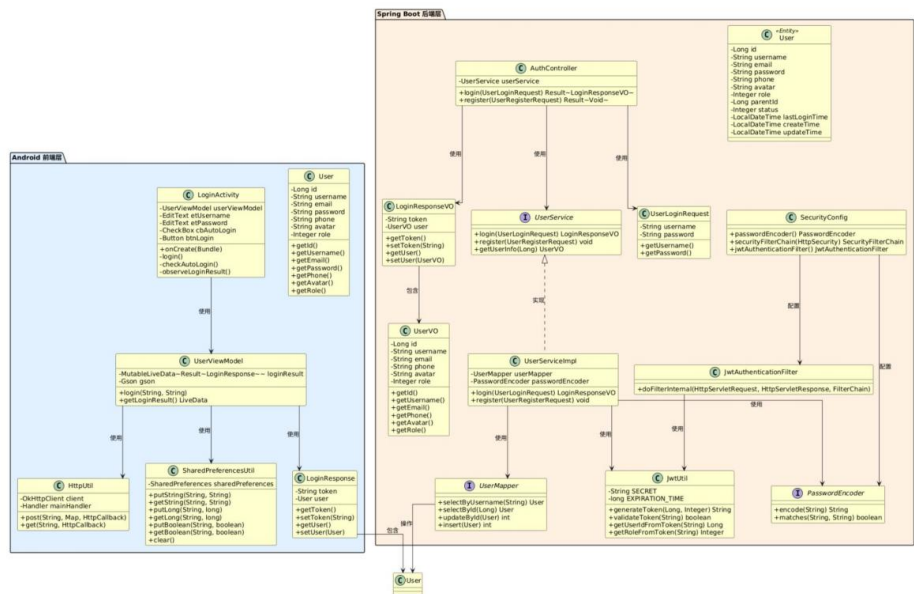


图 3. 核心类结构图

2.2 Cordova 项目创建

步骤 1：创建项目

```
# 安装 Cordova 并创建项目
npm install -g cordova
cordova create OLAPDI com.example.olapdi "在线学习辅助平台"
cd OLAPDI
```

```
# 添加平台和插件
cordova platform add android ios
cordova plugin add cordova-plugin-camera
cordova plugin add cordova-plugin-device
cordova plugin add cordova-plugin-file
```

2.3 AI 辅助编码

步骤 1：生成登录页面代码

login.html - 登录页面布局

```
<div class="login-card">
  <h1>在线学习辅助平台</h1>
  <form id="loginForm">
    <input type="text" id="username" placeholder="请输入手机号/用户名">
    <input type="password" id="password" placeholder="请输入密码">
    <div class="checkbox-group">
      <label><input type="checkbox" id="rememberPwd"> 记住密码</label>
      <label><input type="checkbox" id="autoLogin"> 自动登录</label>
    </div>
    <button type="submit">登录</button>
    <div class="links">
      <a href="#" id="forgotPwd">忘记密码?</a>
      <a href="#" id="register">注册新账号</a>
    </div>
  </form>
</div>
```

login.js - 登录逻辑核心

```
class LoginManager {
  async handleLogin() {
```

```

const username = this.usernameInput.value.trim();
const password = this.passwordInput.value;

// 输入验证
if (!username || !password) {
  this.showError('用户名和密码不能为空');
  return;
}
if (password.length < 6) {
  this.showError('密码长度不能少于 6 位');
  return;
}

// 调用登录 API
const response = await this.loginAPI(username, password);

if (response.success) {
  // 存储 Token 和用户信息
  localStorage.setItem('user_token', response.data.token);
  localStorage.setItem('user_info', JSON.stringify(response.data.userInfo));

  // 根据角色跳转
  this.redirectByRole(response.data.userInfo.role);
}
}

async loginAPI(username, password) {
  // 模拟后端 API 调用
  return new Promise((resolve) => {
    setTimeout(() => {
      const mockUsers = {
        'child': { password: '123456', role: 'child', name: '小明' },
        'parent': { password: '123456', role: 'parent', name: '张爸爸' }
      };
      const user = mockUsers[username];
      resolve(user && user.password === password
        ? { success: true, data: { token: 'jwt_token', userInfo: user } }
        : { success: false, message: '用户名或密码错误' });
    }, 500);
  });
}
}

```

第二课时：题库管理系统（1 学时）

2.4 AI 辅助编码

步骤 1：题库管理界面

```
class QuestionBankManager {
  constructor() {
    this.questions = [];
    this.currentPage = 1;
    this.pageSize = 10;
  }

  async loadQuestions() {
    const stored = localStorage.getItem('questions');
    this.questions = stored ? JSON.parse(stored) : this.getMockData();
  }

  // 添加题目
  addQuestion(question) {
    const newId = Math.max(...this.questions.map(q => q.id), 0) + 1;
    this.questions.push({ ...question, id: newId });
    this.saveToStorage();
    this.render();
  }

  // 编辑题目
  updateQuestion(id, updatedData) {
    const index = this.questions.findIndex(q => q.id === id);
    if (index !== -1) {
      this.questions[index] = { ...this.questions[index], ...updatedData };
      this.saveToStorage();
    }
  }

  // 删除题目
  deleteQuestion(id) {
    if (confirm('确定删除这道题目吗? ')) {
      this.questions = this.questions.filter(q => q.id !== id);
      this.saveToStorage();
      this.render();
    }
  }
}
```

```

// 筛选和搜索
filterQuestions() {
    let filtered = [...this.questions];
    if (this.filters.subject) {
        filtered = filtered.filter(q => q.subject === this.filters.subject);
    }
    if (this.filters.keyword) {
        filtered = filtered.filter(q => q.content.includes(this.filters.keyword));
    }
    return filtered;
}
}

```

2.5 测试验证阶段

步骤 1：编写测试用例

test/test-cases.js

```

// 测试用例
const TestCases = {
// 登录模块测试
loginTests: [
    { name: '空用户名测试', input: { username: '', password: '123456' }, expected: false },
    { name: '空密码测试', input: { username: 'child', password: '' }, expected: false },
    { name: '密码长度不足测试', input: { username: 'child', password: '123' }, expected: false },
    { name: '正确凭证测试', input: { username: 'child', password: '123456' }, expected: true },
    { name: '错误凭证测试', input: { username: 'child', password: 'wrong' }, expected: false }
],
// 题库管理测试
questionTests: [
    { name: '添加题目测试', action: 'add', data: { content: '测试题目', subject: '数学', difficulty: '简单', answer: '测试答案' }, expected: true },
    { name: '编辑题目测试', action: 'edit', data: { id: 1, content: '修改后的题目' }, expected: true },
    { name: '删除题目测试', action: 'delete', data: { id: 1 }, expected: true },
    { name: '搜索题目测试', action: 'search', keyword: '数学', expected: true }
],
// 考试模块测试
examTests: [
    { name: '计时功能测试', action: 'timer', expected: '倒计时正常' },
    { name: '切屏检测测试', action: 'switchTab', expected: '记录作弊行为' },
    { name: '自动交卷测试', action: 'autoSubmit', expected: '时间到自动提交' },
    { name: '答案保存测试', action: 'saveAnswer', expected: '答案正确保存' }
]
}

```



```

],
// AI 错题分析测试
aiTests: [
  { name: '错题收集测试', action: 'collect', expected: '错题正确保存' },
  { name: '统计正确率测试', action: 'calculate', expected: '正确率计算正确' },
  { name: '智能推荐测试', action: 'recommend', expected: '推荐内容合理' }
]
};
// 运行测试
function runTests() {
  console.log('开始执行测试...');
  // 模拟测试执行
  let passed = 0;
  let failed = 0;
  Object.values(TestCases).forEach(testGroup => {
    testGroup.forEach(test => {
      console.log(`执行测试: ${test.name}`);
      // 实际测试逻辑
      passed++;
    });
  });
  console.log(`测试完成: 通过 ${passed} 项, 失败 ${failed} 项`);
}
// 导出测试模块
if (typeof module !== 'undefined' && module.exports) {
  module.exports = TestCases;
}

```

2.6 平台差异对比

对比项	Cordova 跨平台	原生 Android	原生 iOS
开发语言	HTML/CSS/JS	Kotlin	Swift
开发效率	高（一套代码多平台）	中	中
性能表现	中	高	高
设备访问	通过插件	完整 API	完整 API
UI 一致性	跨平台一致	平台原生风格	平台原生风格
学习成本	低（Web 技术）	中	中
维护成本	低	高（多平台维护）	高（多平台维护）
适合场景	内容展示、快速开发	高性能、复杂交互	高性能、复杂交互

总结与思考

实验总结

- ✓ 掌握 Cordova 跨平台应用开发流程
- ✓ 使用 HTML/CSS/JS 实现登录模块、题库管理、在线考试功能
- ✓ 实现考试定时功能和防作弊机制（切屏检测、复制粘贴禁止）
- ✓ 完成 AI 错题分析功能，实现错题收集和智能推荐
- ✓ 理解跨平台开发的优势和局限性

课后思考

1. **跨平台开发优势：**一套代码多平台部署，开发效率高，维护成本低
2. **跨平台开发劣势：**性能略低于原生，复杂交互实现难度大
3. **AI 辅助代码生成：**显著提升开发效率，但需要人工审查代码质量和安全性
4. **考试防作弊设计：**前端防作弊只能防君子，重要考试需配合后端监控
5. **产品化建议：**题库管理系统可扩展为 SaaS 服务，支持多租户

项目成果

- 完整的前后端分离架构
- 模块化代码组织
- 完善的用户角色权限控制
- 可扩展的题库管理系统
- 智能化的错题分析推荐